Anexos ilustrativos al proyecto de trabajo

Gerente de Análisis Monetario (426 05)

Máximo Sangiácomo

Estos anexos son **material ilustrativo, fuera del límite de cuatro páginas del proyecto**; el núcleo se entiende sin ellos. Muestran tres rutinas Python (`pandas`, `statsmodels`) que concretan propuestas de los Ejes 1 y 4. El código completo, con pruebas automáticas (`pytest`), está en la carpeta `codigo/`. **Todas las cifras que siguen provienen de datos sintéticos de demostración, no oficiales**: no son datos del BCRA ni anticipan resultados.

Anexo A. Pronóstico diario de liquidez (Eje 1)

Responde a: Eje 1, funciones 1 y 5. Propuesta: ensamble con efectos de calendario contra un benchmark ingenuo, medido por error absoluto medio relativo, sesgo por día del mes, cobertura de intervalos y test de Diebold-Mariano al 10 % con al menos 60 días. **Método**: se pronostica a un día la variación diaria (%) de la serie. Hay dos modelos, una regresión con variables de calendario (inicio y fin de mes, quincena, aguinaldo, diciembre, lunes) y un ARIMAX (AR(1) en espacio de estado), y el ensamble es su promedio simple. El benchmark es la variación nula. La evaluación es fuera de muestra con ventana expansiva: los parámetros se reestiman cada 60 días, sin *look-ahead*. El test de Diebold-Mariano lleva la corrección de Harvey, Leybourne y Newbold (HLN).

```
# Fragmento abreviado de codigo/monetario_demo/liquidez.py
for t0 in range(n0, len(y), cada):             # ventana expansiva
    t1 = min(t0 + cada, len(y))
    y_tr, X_tr, X_te, obs = y.iloc[:t0], X.iloc[:t0], X.iloc[t0:t1], y.iloc[t0:t1]
    q = float(np.quantile(np.abs(y_tr), 1 - ALFA))          # benchmark ingenuo
    salida["ingenuo"].append(pd.DataFrame({"obs": obs, "pron": 0.0, "inf": -q, "sup": q}))
    pr = sm.OLS(y_tr, X_tr).fit().get_prediction(X_te).summary_frame(alpha=ALFA)
    ...                                        # regresión con calendario
    res = SARIMAX(y_tr.to_numpy(), exog=ex_tr, order=(1, 0, 0), trend="c").fit(disp=False)
    pa = res.append(obs.to_numpy(), exog=ex_te).get_prediction(start=t0, end=t1 - 1)
    ...                                        # ARIMAX: filtrado un paso adelante
out["ensamble"] = (out["regresion"] + out["arimax"]) / 2
dm = diebold_mariano(e_ens, e_ing, h=1, perdida="abs", alternativa="menor")  # HLN
```

Uso: `python -m monetario_demo.liquidez --demo` (con datos reales: `--csv <serie> --ultimos 60`). **Salida demostrativa: datos sintéticos de demostración, no oficiales.**

```
Evaluación fuera de muestra, 1 día, n = 782
             MAE  MAE_rel  sesgo  cobertura
ingenuo    0.848    1.000  0.301      0.882
regresion  0.662    0.780  0.071      0.885
arimax     0.629    0.742  0.055      0.895
ensamble   0.640    0.755  0.063      0.896
DM-HLN ensamble vs. ingenuo (pérdida abs., H1: ensamble mejor): est. = -11.31, p = 0.000
  -> rechaza igualdad al 10 % (n >= 60: cumple)
Sesgo medio por posición en el mes (pp):   ensamble  ingenuo
  fin de mes                                  0.048    1.689
  inicio de mes                              -0.030   -0.725
```

Lectura: con estos datos sintéticos, el ensamble reduce el error absoluto medio un 24,5 % frente al ingenuo y corrige el sesgo de fin de mes. Sus intervalos del 90 % cubren el 89,6 %.

Anexo B. Demanda de dinero en ventanas móviles (Eje 4)

Responde a: Eje 4, función 4, primer bloque de la programación. Propuesta: modelo de corrección de errores en ventanas móviles para la base y el M2 privado. **Método**: Engle-Granger en dos pasos con $m - p$ (dinero real), y (actividad, EMAE) e i (costo de oportunidad), en ventanas de 72 meses. Una alerta indica que b_y o b_i se alejó de la primera ventana más que un umbral simulado bajo estabilidad: es un *bootstrap* paramétrico con error AR(1) corregido por sesgo, con falsa alarma nominal del 10 % por coeficiente en toda la muestra. Se informa además el CUSUM del modelo de corto plazo.

```
# Fragmento abreviado de codigo/monetario_demo/demanda_dinero.py (umbral_simulado y alerta)
beta = np.linalg.lstsq(X[:ventana], m[:ventana], rcond=None)[0]   # ventana de referencia
u = m[:ventana] - X[:ventana] @ beta
...                                       # phi del AR(1) de u, corregido por sesgo
maximos = np.empty((B, 2))
for b in range(B):                        # B muestras sin quiebre
    us = _simular_u(phi, sigma, len(df))
    bm = _betas_moviles(X @ beta + us, X, ventana)[:, 1:]
    maximos[b] = np.abs(bm - bm[0]).max(axis=0)
q = np.quantile(maximos, nivel, axis=0)    # umbral de b_y y b_i
...
fuera = (rol[coef] - rol[coef].iloc[0]).abs() > umbral[coef]
out[coef] = fuera.idxmax() if fuera.any() else None   # primera alerta
```

Uso: `python -m monetario_demo.demanda_dinero --demo`. **Salida demostrativa: datos sintéticos de demostración, no oficiales.** El generador incluye un quiebre de b_i (de -2 a -4) en 2016-01.

```
Muestra completa: b_y = 0.24, b_i = -2.19, p Engle-Granger = 0.387, ajuste g = -0.072 (t = -2.7)
CUSUM del MCE: p = 0.632
Ventanas de 72 meses:  primera 2009-12: b_i = -1.99, p EG = 0.004, g = -0.748
                       última  2024-12: b_i = -4.12, p EG = 0.005, g = -0.452
Umbral simulado bajo estabilidad (90 %): b_y +/- 0.54, b_i +/- 0.30
Alerta de inestabilidad (primer fin de ventana fuera del umbral): b_y: 2016-03; b_i: 2016-02
```

Lectura: la estimación con la muestra completa pierde la cointegración ($p = 0{,}39$) y el CUSUM no detecta el cambio, porque mira el corto plazo. Las ventanas móviles dan la alerta un mes después del quiebre.

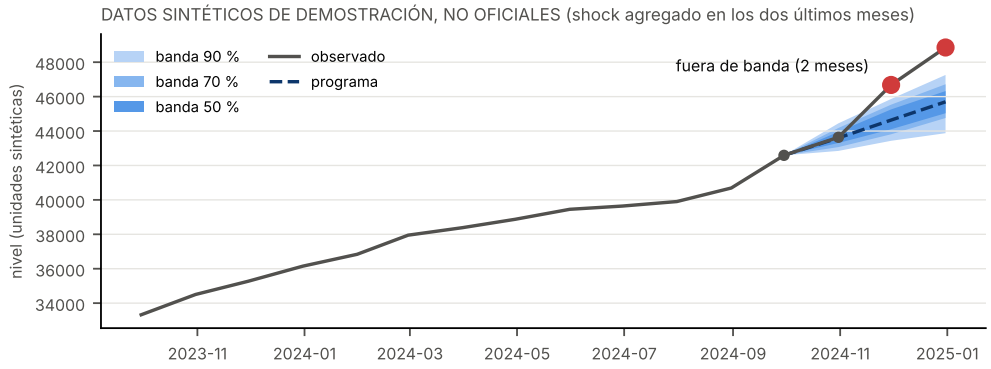
Anexo C. Banda de tolerancia y desvíos (Eje 4)

Responde a: Eje 4, función 4, tercer bloque de la programación, e indicador (vi) del proyecto. Propuesta: banda de tolerancia para base y M2 derivada del error histórico, revisada cada mes, con nota si se excede. **Método**: errores históricos (%) del programa a 1–3 meses. La banda central del 90 % sale de sus cuantiles empíricos y usa sólo errores ya observados al armar cada programa. Cada mes del trimestre queda clasificado como *dentro* o *fuera de banda*.

```
# Fragmento abreviado de codigo/monetario_demo/bandas.py (banda_tolerancia y chequeo)
pos_corte = errores.index.get_indexer([corte], method="pad")[0]
for h in errores.columns:                 # h = 1, 2, 3 meses
    e = errores[h].iloc[: max(pos_corte - h + 1, 0)].dropna()   # conocidos al corte
    out[h] = {f"{lado}_{int(n * 100)}": float(np.quantile(e, q))
              for n in niveles for lado, q in (("inf", (1 - n) / 2), ("sup", (1 + n) / 2))}
...
desvio = 100 * (lg.iloc[t + h] - prog)     # observado frente a programa (%)
estado = "dentro de banda" if inf <= desvio <= sup else "fuera de banda"
```

Uso: `python -m monetario_demo.bandas --demo --figura salidas/fan_chart_demo.pdf`. **Salida demostrativa: datos sintéticos de demostración, no oficiales.** Se agregó un *shock* sintético en los dos últimos meses.

mes	h	desvio_%	inf_%	sup_%	estado
2024-10	1	0.1	-1.7	1.9	dentro de banda
2024-11	2	4.5	-2.7	2.7	fuera de banda
2024-12	3	6.7	-4.1	3.4	fuera de banda



Fan chart (bandas del 50, 70 y 90 %) del último trimestre. Datos sintéticos de demostración, no oficiales.

Nota: paso a datos reales

1. Descargar series públicas con `codigo/descargar_bcra.py`, que usa la API de Estadísticas Monetarias v4.0 del BCRA. Sirve para buscar los IDs en el catálogo y bajar cada serie con su metodología a `datos/` en formato `fecha,valor`. El EMAE y el IPC se bajan de INDEC. La guía está en `datos/LEEME.md`.
2. Correr las mismas rutinas con `--csv` (u otros argumentos) en lugar de `--demo`. Revisar primero, con el área, el calendario de feriados y del período de cómputo, la definición de M2 privado y el costo de oportunidad.
3. Antes de usar los resultados, validar contra el proceso vigente y correr en paralelo; en el pronóstico de liquidez, al menos 60 días. Toda serie interna o por entidad queda en el entorno interno.